

Team Name: Books.gov

Sean Gilday  
Gavin McIlvaine  
Elan Cote  
Nathaniel Loomis  
Jay Stebbins

April 17, 2025

## 1 Introduction

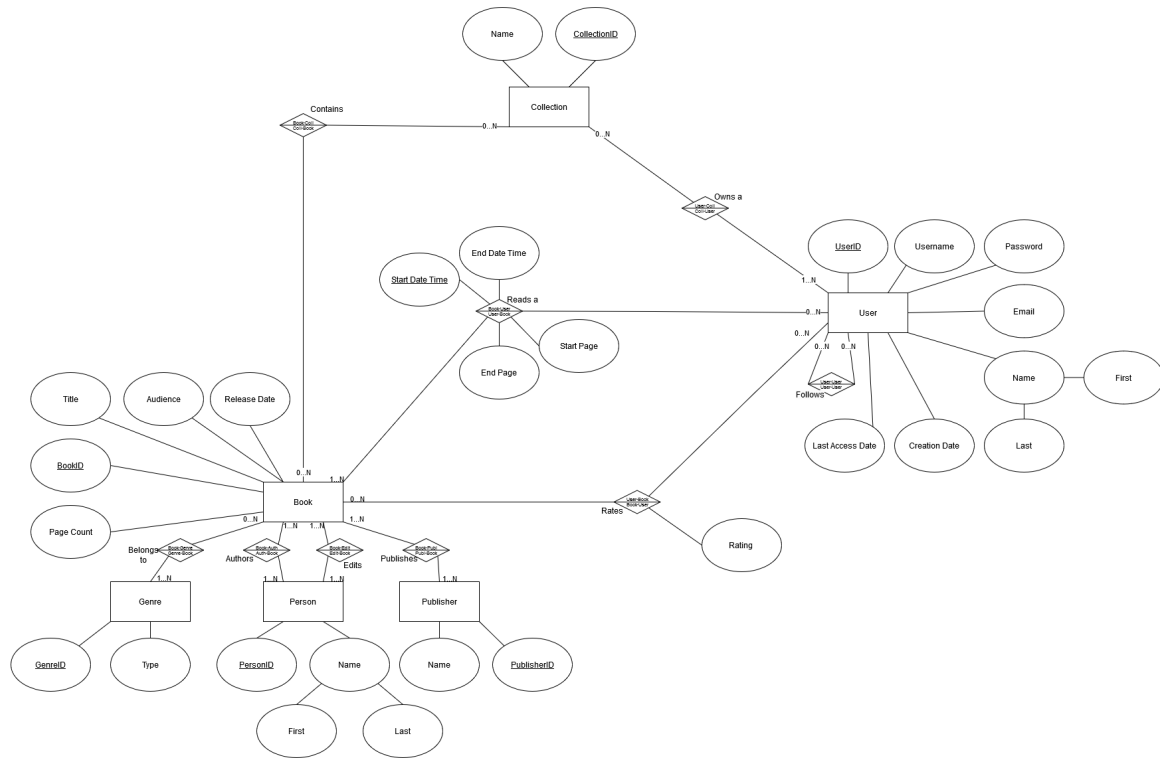
The database we are designing is meant to manage a user's reading habits and interactions. This database stores essential information about users, books, and interactions between the two. The system stores essential user information such as username, a securely stored password, first and last name, email, account creation date, and last access date. This information allows multiple users to use the system effectively. The database also manages book data, including title, intended audience (Kids, Teens, or Adults), release date, length in pages, genres, authors, editors, and publishers. In addition, the database will store unique identifiers and names of authors, editors, and publishers to ensure accurate record-keeping.

Users can take several actions within the system that get stored. When a user reads a book, the system stores the sessionID, start and end pages, date, and start and end time. Users can also rate books from 1-5, which will subsequently be stored and used to calculate an average rating. Finally, users can create and manage collections of books as well as follow and be followed by other users.

Our team will develop this database in Java and SQL and use Discord to communicate with each other.

## 2 Design

### 2.1 Conceptual Model



During the development of our EER diagram, we had many different entities and relationships to consider. Initially, we identified a rating and reading session as separate entities created by the user. However, after further consideration, we realized that those two describe the relationships between a user and a book. A user reads a book and records the start and end date and page, and a user rates a book, storing the rating. At the beginning of the development of this diagram, we had genre as an attribute of a book, but due to the wide domain of a genre, we opted to put it as its own entity with an id and the genre type as an attribute. This will allow a large domain of genres to be input into the database and allows easy access to the many different types. To consolidate our entities, we combined the editor and author entities into a single-person entity with two different relationships with a book. This streamlines the author and editor data into a single location to store person data and makes it easy to access both relations at once.

### 2.2 Reduction to Tables

User, Book, Collection, Author, Editor, Publisher, and Genre are all entity types that warrant their own entry in the relational model. For every many-to-many relationship in the EER diagram, an entry in the relational model is needed to link the entities that contain their respective primary keys. The entries relating to this are: User\_Follows, User\_Collection, Collection\_Book, Book\_Author, Book\_Editor, Book\_Publisher, Book\_Genre, User\_Book\_Rating, User\_Book\_Session.

User (User\_ID, Username, Password, First\_Name, Last\_Name, Email, Creation\_Date, Last\_Access\_Date)

User\_Follows (User\_Follower, User\_Followed)

User\_Collection (User\_ID, Collection\_ID)

Book (Book\_ID, Title, Audience, Length, Release\_Date)

Collection (Collection\_ID, User\_ID, Name)

Collection\_Book (Collection\_ID, Book\_ID)

Author (Person\_ID, First\_Name, Last\_Name)

Book\_Author (Book\_ID, Person\_ID)

Editor (Person\_ID, First\_Name, Last\_Name)  
Book\_Editor (Book\_ID, Person\_ID)  
Publisher (Publisher\_ID, Name)  
Book\_Publisher (Book\_ID, Publisher\_ID)  
Genre (Genre\_ID, Genre\_Type)  
Book\_Genre (Book\_ID, Genre\_ID)  
User\_Book\_Rating (User\_ID, Book\_ID, Rating)  
User\_Book\_Session (User\_ID, Book\_ID, Start\_Time, End\_Time, Start\_Page, End\_Page)

## 2.3 Data Requirements/Constraints

The project database will store essential information to support the application requirements. It will manage multiple users by recording details such as username, password (secured in the final phase), first and last name, email, creation date, and last access date. For books, the database will capture the title, audience category (Kids, Teens, Adults), release date, length in pages, genres, authors, editors, and publishers. In addition, authors, editors, and publishers will be uniquely identifiable by their names. The system will also track user actions, including reading sessions (pages or locations read, start and end times, and date), star ratings for books (1-5, fixed per user per book), creation of named book collections, and user interactions like following and user following.

User:

User\_ID is a primary key  
User\_ID, Username, Password, First\_Name, Last\_Name, Email, Creation\_Date, Last\_Access\_Date must be NOT NULL  
UserID, Username must be unique  
Last\_Access\_Date must not be before Creation\_Date  
Users do not need to follow anyone, be followed by anyone, own any collections, rate any books, nor have any reading sessions  
User\_ID is a number  
Username, Password, First\_Name, Last\_Name, and Email are strings  
Creation\_Date and Last\_Access\_Date are dates

Book:

Book\_ID is a primary key  
Book\_ID, Title, Audience, Length, Release\_Date must be NOT NULL  
Book\_ID must be unique  
Books must have an author, editor, publisher, and genre  
Books do not need to be in a collection, have a rating, or be in any reading sessions.  
Audience must be either Kids, Teens, or Adults.  
Book\_ID, Length are numbers  
Title, Audience are strings  
Release\_Date is a date

Collection:

Collection\_ID is a primary key  
User\_ID is a foreign key from the User table  
Collection\_ID, User\_ID, Name must be NOT NULL  
Collection\_ID, User\_ID must be unique  
Collection\_ID and User\_ID are numbers  
Name is a string

Author: Person\_ID is a primary key

Person\_ID, First\_Name, Last\_Name must be NOT NULL  
Person\_ID must be unique  
Person\_ID is a number

First\_Name and Last\_Name are strings

Editor:

Person\_ID is a primary key

Person\_ID, First\_Name, Last\_Name must be NOT NULL

Person\_ID must be unique

Person\_ID is a number

First\_Name and Last\_Name are strings

Publisher:

Publisher\_ID is a primary key

Publisher\_ID, Name must be NOT NULL

Publisher\_ID must be unique

Publisher\_ID is a number

Name is a strings

Genre:

Genre\_ID is a primary key

Genre\_ID, Genre\_Type must be NOT NULL

Genre\_ID, Genre\_Type must be unique

Genre\_ID is a number Genre\_Type is a string

User\_Follows:

The aggregate of User\_Follower, User\_Followed is the primary key

User\_Follower, User\_Followed are foreign keys from the User table

User\_Follower, User\_Followed must be NOT NULL and be numbers

User\_Collection:

The aggregate of User\_ID, Collection\_ID is the primary key

User\_ID is a foreign key from the User table

Collection\_ID is a foreign key from the Collection table

User\_ID, Collection\_ID must be NOT NULL and be numbers

Collection\_Book:

The aggregate of Collection\_ID, Book\_ID is the primary key

Collection\_ID is a foreign key from the Collection table

Book\_ID is a foreign key from the Book table

Collection\_ID, Book\_ID must be NOT NULL and be numbers

Book\_Author:

The aggregate of Book\_ID, Person\_ID is the primary key

Book\_ID is a foreign key from the Book table

Person\_ID is a foreign key from the Person table

Book\_ID, Person\_ID must be NOT NULL and be numbers

Book\_Editor:

The aggregate of Book\_ID, Person\_ID is the primary key

Book\_ID is a foreign key from the Book table

Person\_ID is a foreign key from the Person table

Book\_ID, Person\_ID must be NOT NULL and be numbers

Book\_Publisher:

The aggregate of Book\_ID, Publisher\_ID is the primary key

Book\_ID is a foreign key from the Book table

Publisher\_ID is a foreign key from the Publisher table

Book\_ID, Publisher\_ID must be NOT NULL and be numbers

Book\_Genre:

The aggregate of Book\_ID, Genre\_ID is the primary key

Book\_ID is a foreign key from the Book table

Genre\_ID is a foreign key from the Genre table

Book\_ID, Genre\_ID must be NOT NULL and be numbers

User\_Book\_Rating:

The aggregate of User\_ID, Book\_ID is the primary key

User\_ID is a foreign key from the User table

Book\_ID is a foreign key from the Book table

User\_ID, Book\_ID, Rating must be NOT NULL and be numbers

Rating must be between 1 and 5 (inclusive)

User\_Book\_Session:

The aggregate of Start\_Time, Book\_ID, and User\_ID is the primary Key

Book\_ID and User\_ID, Book are foreign keys from tables Book and User

Start\_Time, User\_ID, Book\_ID, End\_Time, Start\_Page, End\_Page must be NOT NULL

Start\_Time, End\_Time must be valid date formats

End\_Time must not be before Start\_Time

Start\_Page must be non-negative and End\_Page must not be less than Start\_Page

Book\_ID, User\_ID, Start\_Page, and End\_Page are numbers

Start\_Time and End\_Time are dates

## 2.4 Sample instance data

| User_ID | Username       | Password   | First_Name | Last_Name | Email                   | Creation Date | Last Access Date |
|---------|----------------|------------|------------|-----------|-------------------------|---------------|------------------|
| 1       | jdoe123        | ma0im43#   | John       | Doe       | johndoe@gmail.com       | 2023-1-10     | 2024-1-15        |
| 2       | JaneSmith22    | g03MRQ@#   | Jane       | Smith     | janesmith@gmail.com     | 2022-5-20     | 2024-1-18        |
| 3       | AliceTheReader | 03rnwpe#   | Alice      | Jones     | alicejones@gmail.com    | 2021-7-15     | 2024-1-10        |
| 4       | BobbiBobbi     | U(.3rh3a   | Bob        | Brown     | bobbrown@gmail.com      | 2023-12-22    | 2023-12-20       |
| 5       | Charlie03      | #10\$Nknew | Charlie    | Wilson    | charliewilson@gmail.com | 2019-11-22    | 2024-1-28        |

Table 1: User Data Table

| Book_ID | Title               | Audience | Length | Release_Date |
|---------|---------------------|----------|--------|--------------|
| 1       | The Great Adventure | Adults   | 320    | 2021-05-10   |
| 2       | Science Wonders     | Teens    | 250    | 2022-08-14   |
| 3       | Fantasy Kingdom     | Kids     | 180    | 2023-01-30   |
| 4       | History of Time     | Adults   | 400    | 2018-11-30   |
| 5       | Mystery of the Lost | Teens    | 290    | 2020-04-15   |

Table 2: Book Data Table

| Collection_ID | User_ID | Name                       |
|---------------|---------|----------------------------|
| 1             | 1       | John's Favorite Reads      |
| 2             | 2       | Jane's Science Picks       |
| 3             | 3       | Alice's Fantasy Collection |
| 4             | 4       | Bob's History Books        |
| 5             | 5       | Bob's Mystery Reads        |

Table 3: Collection Data Table

| Person_ID | First_Name | Last_Name |
|-----------|------------|-----------|
| 1         | Mark       | Twain     |
| 2         | Isaac      | Newton    |
| 3         | J.K.       | Rowling   |
| 4         | Stephen    | Hawking   |
| 5         | Agatha     | Christie  |

Table 4: Author Data Table

| Person_ID | First_Name | Last_Name |
|-----------|------------|-----------|
| 1         | Emily      | White     |
| 2         | Michael    | Black     |
| 3         | Sarah      | Green     |
| 4         | David      | Blue      |
| 5         | Laura      | Brown     |

Table 5: Editor Data Table

| Publisher_ID | Name                        |
|--------------|-----------------------------|
| 1            | Penguin Books               |
| 2            | HarperCollins               |
| 3            | Oxford Press                |
| 4            | American Library Collection |
| 5            | Random House                |

Table 6: Publisher Data Table

| Genre_ID | Genre_Type |
|----------|------------|
| 1        | Adventure  |
| 2        | Science    |
| 3        | Fantasy    |
| 4        | History    |
| 5        | Mystery    |

Table 7: Genre Data Table

| User_Follower | User_Followed |
|---------------|---------------|
| 1             | 2             |
| 1             | 3             |
| 2             | 3             |
| 3             | 2             |
| 4             | 3             |

Table 8: User Follows Table

| Collection_ID | Book_ID |
|---------------|---------|
| 1             | 1       |
| 2             | 2       |
| 3             | 3       |
| 4             | 4       |
| 5             | 5       |

Table 9: Collection Book Table

| Book_ID | Person_ID |
|---------|-----------|
| 1       | 1         |
| 2       | 2         |
| 3       | 3         |
| 4       | 4         |
| 5       | 5         |

Table 10: Book Author Table

| Book_ID | Person_ID |
|---------|-----------|
| 1       | 1         |
| 2       | 2         |
| 3       | 3         |
| 4       | 4         |
| 5       | 5         |

Table 11: Book Editor Table

| Book_ID | Publisher_ID |
|---------|--------------|
| 1       | 1            |
| 2       | 2            |
| 3       | 3            |
| 4       | 4            |
| 5       | 5            |

Table 12: Book Publisher Table

| Book_ID | Genre_Type |
|---------|------------|
| 1       | 1          |
| 2       | 2          |
| 3       | 3          |
| 4       | 4          |
| 5       | 5          |

Table 13: Book Genre Table

| User_ID | Book_ID | Rating |
|---------|---------|--------|
| 1       | 1       | 5      |
| 2       | 2       | 4      |
| 3       | 3       | 4      |
| 4       | 4       | 3      |
| 5       | 5       | 2      |

Table 14: User Book Rating Table

| Start_Time          | User_ID | Book_ID | End_Time            | Start_Page | End_Page |
|---------------------|---------|---------|---------------------|------------|----------|
| 2024-01-01 10:00:00 | 1       | 1       | 2024-01-01 11:00:00 | 1          | 50       |
| 2024-01-02 14:00:00 | 2       | 2       | 2024-01-02 15:00:00 | 1          | 2        |
| 2024-01-03 16:00:00 | 3       | 3       | 2024-01-03 17:00:00 | 1          | 31       |
| 2024-01-04 09:00:00 | 4       | 4       | 2024-01-04 10:00:00 | 1          | 27       |
| 2024-01-05 13:00:00 | 5       | 5       | 2024-01-05 14:00:00 | 1          | 23       |

Table 15: User Book Session Table

## 3 Implementation

### 3.1 Sample SQL Statements

For the overall implementation of our database, we used the relational model and the constraints listed above to help make our tables. We used the SERIAL type for IDs as these are automatically increasing and unique per entry.

Below is an example of how we created the users table.

```
CREATE TABLE "users" (  
    User_ID SERIAL NOT NULL PRIMARY KEY,  
    Username VARCHAR(255) NOT NULL UNIQUE,  
    Password VARCHAR(255) NOT NULL,  
    First_Name VARCHAR(255) NOT NULL,  
    Last_Name VARCHAR(255) NOT NULL,  
    Email VARCHAR(255) NOT NULL,  
    Creation_Date TIMESTAMP NOT NULL,  
    Last_Access_Date TIMESTAMP NOT NULL,  
    CHECK (Last_Access_Date >= Creation_Date)  
);
```

When creating a new user, the SQL insert statement used in our application looked like the following, where the ?'s are each value being inserted. The Creation\_Date and Last\_Access\_Date were generated when the function were run, but the other 5 fields were obtained from the person running the application.

```
"INSERT INTO users (Username, Password, First_Name, Last_Name, Email, Creation_Date, Last_Access_Date)  
VALUES (?, ?, ?, ?, ?, ?, ?)"
```

Below is an example of how we created the collection table.

```
CREATE TABLE "collection" (  
    Collection_ID SERIAL NOT NULL PRIMARY KEY,  
    Name VARCHAR(255) NOT NULL  
);
```

When creating a new collection, the SQL insert statement used in our application looked like the following, where the ?'s are each value being inserted. There is only one ? as there is a separate table where the users table and the collection table are both referenced via foreign keys.

```
"INSERT INTO collection (Name) VALUES (?)"
```

When loading initial data into the database, the team used Mockaroo. The team generated 1000 rows of data at a time and ran the SQL insert statements in DataGrip, which would filter out invalid entries. After we generated data for all of the primary tables, we generated data for the M:N by using data generated from these tables and taking advantage of the RANDOM() function. Below are examples for 5 different tables.

Users:

```
INSERT INTO "users" (Username, Password, First_Name, Last_Name, Email, Creation_Date, Last_Access_Date)  
VALUES ('jdoe', 'hashed_password_1', 'John', 'Doe', 'jdoe@email.com', '2024-01-01 12:00:00', '2024-  
03-01 15:30:00')
```

```
Book: INSERT INTO "book" (Title, Audience, Length, Release_Date) VALUES ('The Great Ad-  
venture', 'Teens', 350, '2023-05-15 00:00:00')
```

```
Author: INSERT INTO "author" (First_Name, Last_Name) VALUES ('Mark', 'Twain')
```

Publisher: INSERT INTO "publisher" (Name) VALUES ('Penguin Random House')

Genre: INSERT INTO "genre" (Genre\_Type) VALUES ('Fiction')

## 3.2 Appendix

Below are a list of the SQL statements used during Phase 3:

Query for our hypothesis of books with less pages will have a higher average rating:

```
SELECT
    CONCAT(FLOOR(b.length / 50) * 50 + 1, '-', FLOOR(b.length / 50) * 50 + 50) AS page_range,
    AVG(ubr.rating) AS avg_rating
FROM book AS b
JOIN user_book_rating AS ubr ON b.book_id = ubr.book_id
GROUP BY page_range
ORDER BY avg_rating DESC;
```

Query for our hypothesis of during the summer months, books with more authors will have a higher average rating:

```
SELECT num_authors,
    AVG(ubr.rating) AS avg_rating
FROM (
    SELECT COUNT(ba.person_id) AS num_authors,
        b.book_id
    FROM book AS b
    JOIN book_author AS ba ON b.book_id = ba.book_id
    JOIN user_book_rating AS ubr ON b.book_id = ubr.book_id
    JOIN user_book_session AS ubs ON ubr.user_id = ubs.user_id AND ubr.book_id = ubs.book_id
    WHERE EXTRACT(month FROM ubs.start_time) IN (6, 7, 8) -- Summer months: June, July, August
    GROUP BY b.book_id
) AS book_stats
JOIN user_book_rating AS ubr ON book_stats.book_id = ubr.book_id
GROUP BY num_authors
ORDER BY num_authors DESC;
```

Query for displaying the number of users following this user and the number of users this user is following, where the ? is the user's id:

```
SELECT * FROM user_follows WHERE User_Followed = ?;
SELECT * FROM user_follows WHERE User_Follower = ?;
```

Query for displaying the top 20 most popular books in the last 90 days:

```
WITH session_stats AS (
    SELECT
        Book_ID,
        COUNT(*) AS reading_session_count
    FROM
        User_Book_Session
    WHERE
        Start_Time >= CURRENT_DATE - INTERVAL '90 days'
    GROUP BY
        Book_ID
),
rating_stats AS (
    SELECT
```

```

        Book_ID,
        AVG(Rating) AS average_rating
    FROM
        User_Book_Rating
    GROUP BY
        Book_ID
),
combined AS (
    SELECT
        ss.Book_ID,
        ss.reading_session_count,
        rs.average_rating
    FROM
        session_stats ss
    JOIN
        rating_stats rs ON ss.Book_ID = rs.Book_ID
),
normalized AS (
    SELECT
        *,
        (reading_session_count * 1.0 - MIN(reading_session_count) OVER ()) /
        NULLIF((MAX(reading_session_count) OVER () - MIN(reading_session_count) OVER ()), 0)
        AS normalized_sessions,

        (average_rating - MIN(average_rating) OVER ()) /
        NULLIF((MAX(average_rating) OVER () - MIN(average_rating) OVER ()), 0)
        AS normalized_rating
    FROM
        combined
)
SELECT
    n.Book_ID,
    b.Title,
    n.reading_session_count,
    n.average_rating,
    ROUND(0.6 * normalized_rating + 0.4 * normalized_sessions, 4) AS composite_score
FROM
    normalized n
JOIN
    Book b ON n.Book_ID = b.Book_ID
ORDER BY
    composite_score DESC
LIMIT 20;

```

Query for displaying the top 20 most popular books among users followed by the current user, where the first ? is the user's id and the second is the limit of entries returned (20)

```

SELECT ubr.Book_ID, b.title, " +
    "STRING_AGG(DISTINCT CONCAT(a.first_name, ' ', a.last_name), ', ') AS authors, " +
    "p.name AS publisher, " +
    "b.length, " +
    "b.audience, " +
    "COALESCE(AVG(ubr.Rating), 0) AS avg_rating, " +
    "EXTRACT(YEAR FROM b.release_date) AS release_year, " +
    "COALESCE(STRING_AGG(DISTINCT g.genre_type, ', '), 'Unknown') AS genre " +
    "FROM user_book_rating ubr " +
    "JOIN user_follows uf ON ubr.User_ID = uf.User_Followed " +
    "JOIN book b ON ubr.Book_ID = b.book_id " +

```

```

"LEFT JOIN book_author ba ON b.book_id = ba.book_id " +
"LEFT JOIN author a ON ba.person_id = a.person_id " +
"LEFT JOIN book_publisher bp ON b.book_id = bp.book_id " +
"LEFT JOIN publisher p ON bp.publisher_id = p.publisher_id " +
"LEFT JOIN book_genre bg ON b.book_id = bg.book_id " +
"LEFT JOIN genre g ON bg.genre_id = g.genre_id " +
"WHERE uf.User_Follower = ? " +
"GROUP BY ubr.Book_ID, b.title, p.name, b.length, b.audience, b.release_date " +
"ORDER BY avg_rating DESC " +
"LIMIT ?";

```

Query for displaying the top 5 new releases of the month:

```

WITH new_releases AS (
    SELECT *
    FROM Book
    WHERE DATE_TRUNC('month', Release_Date) = DATE_TRUNC('month', CURRENT_DATE)
),
book_reads AS (
    SELECT
        Book_ID,
        COUNT(*) AS total_reads
    FROM User_Book_Session
    GROUP BY Book_ID
),
book_ratings AS (
    SELECT
        Book_ID,
        AVG(Rating) AS avg_rating
    FROM User_Book_Rating
    GROUP BY Book_ID
),
combined AS (
    SELECT
        nr.Book_ID,
        nr.Title,
        nr.release_date,
        COALESCE(br.total_reads, 0) AS total_reads,
        COALESCE(r.avg_rating, 0) AS avg_rating
    FROM new_releases nr
    LEFT JOIN book_reads br ON nr.Book_ID = br.Book_ID
    LEFT JOIN book_ratings r ON nr.Book_ID = r.Book_ID
),
normalized AS (
    SELECT
        *,
        (total_reads * 1.0 - MIN(total_reads) OVER ()) /
        NULLIF((MAX(total_reads) OVER () - MIN(total_reads) OVER ()), 0)
        AS normalized_sessions,

        (avg_rating - MIN(avg_rating) OVER ()) /
        NULLIF((MAX(avg_rating) OVER () - MIN(avg_rating) OVER ()), 0)
        AS normalized_rating
    FROM
        combined
)
SELECT
    n.Book_ID,

```

```

        b.Title,
        n.release_date,
        n.avg_rating,
        ROUND(0.6 * normalized_rating + 0.4 * normalized_sessions, 4) AS composite_score
FROM
    normalized n
JOIN
    Book b ON n.Book_ID = b.Book_ID
ORDER BY
    composite_score DESC
LIMIT 5;

```

Query for recommending books based on a user's read history, where the first three ?'s are the user's id and the fourth is the limit of entries returned (10):

```

WITH UserPreferences AS (
    SELECT bg.genre_id, ba.person_id AS author_id
    FROM user_book_rating ubr
    JOIN book_genre bg ON ubr.book_id = bg.book_id
    JOIN book_author ba ON ubr.book_id = ba.book_id
    WHERE ubr.user_id = ?
    GROUP BY bg.genre_id, ba.person_id
),
SimilarUsers AS (
    SELECT ubr.user_id,
        COUNT(DISTINCT bg.genre_id) AS genre_match_count,
        COUNT(DISTINCT ba.person_id) AS author_match_count,
        COUNT(DISTINCT ubr.book_id) AS book_match_count,
        COUNT(DISTINCT CASE WHEN ubr.rating >= 4 THEN ubr.book_id END) AS high_rating_match
    FROM user_book_rating ubr
    JOIN book_genre bg ON ubr.book_id = bg.book_id
    JOIN book_author ba ON ubr.book_id = ba.book_id
    WHERE (bg.genre_id IN (SELECT genre_id FROM UserPreferences)
        OR ba.person_id IN (SELECT author_id FROM UserPreferences))
        AND ubr.user_id <> ?
    GROUP BY ubr.user_id
),
RankedUsers AS (
    SELECT user_id,
        (genre_match_count * 2 + author_match_count * 2 + book_match_count + high_rating_match * 3)
    FROM SimilarUsers
    ORDER BY similarity_score DESC
    LIMIT 10
)
SELECT b.book_id, b.title,
    STRING_AGG(DISTINCT CONCAT(a.first_name, ' ', a.last_name), ', ') AS authors,
    p.name AS publisher, b.length, b.audience,
    COALESCE(AVG(ubr.rating), 0) AS avg_rating,
    EXTRACT(YEAR FROM b.release_date) AS release_year,
    COALESCE(STRING_AGG(DISTINCT g.genre_type, ', '), 'Unknown') AS genre
FROM user_book_rating ubr
JOIN book b ON ubr.book_id = b.book_id
LEFT JOIN book_author ba ON b.book_id = ba.book_id
LEFT JOIN author a ON ba.person_id = a.person_id
LEFT JOIN book_publisher bp ON b.book_id = bp.book_id
LEFT JOIN publisher p ON bp.publisher_id = p.publisher_id
LEFT JOIN book_genre bg ON b.book_id = bg.book_id
LEFT JOIN genre g ON bg.genre_id = g.genre_id

```

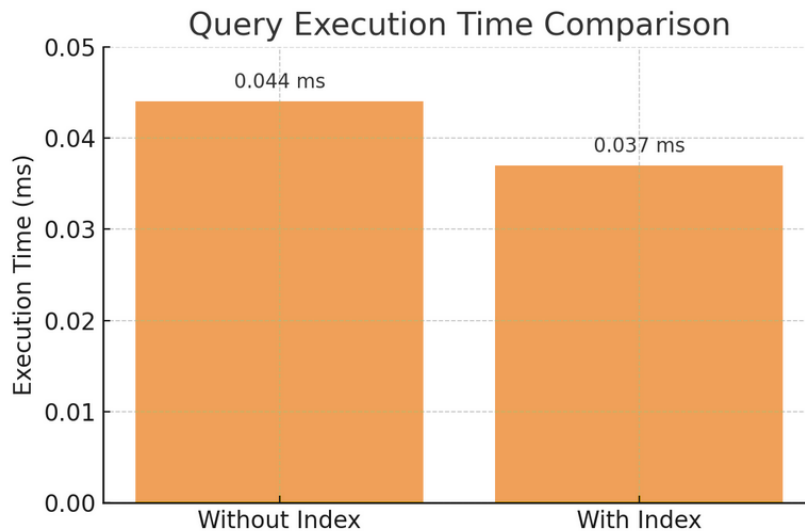
```
WHERE ubr.user_id IN (SELECT user_id FROM RankedUsers)
AND ubr.book_id NOT IN (SELECT book_id FROM user_book_rating WHERE user_id = ?)
GROUP BY b.book_id, b.title, p.name, b.length, b.audience, b.release_date
ORDER BY avg_rating DESC
LIMIT ?;
```

Below is the query used to retrieve the user passwords so the Java code could update salt and hash them and then the query to update the passwords in the database, where the first ? is the hashedPassword and the second ? is the user's id:

```
SELECT user_id, password FROM users";
String updateSql = "UPDATE users SET password = ? WHERE user_id = ?;
```

### 3.3 Index Analysis

Our most robust data section was by far our user\_book\_rating table. This table had over 7,000 entries compared to the 5,000 entries our other tables contained. This would make queries along this table extremely inefficient, which would compromise our methods that involve fetching book ratings, especially return the top X books from a certain group. This is why we decided to create an index for this table. It was created by sorting the user\_id attribute in ascending order because in most of our queries regarding the table that is how we are locating our data. We were able to do so successfully and sped up the process by about 16%.



## 4 Data Analysis

### 4.1 Hypothesis

Books with less pages will have a higher average rating.

During the summer months, books with more authors will have a higher average rating.

### 4.2 Data Pre-processing

The pre-processing steps used to prepare the data for analytics was to generate more data for various cases. These cases included adding more user rating for books users have already read as before, there was a very minimal amount. Furthermore, more data was generated for reading sessions during summer months so that there was more data.

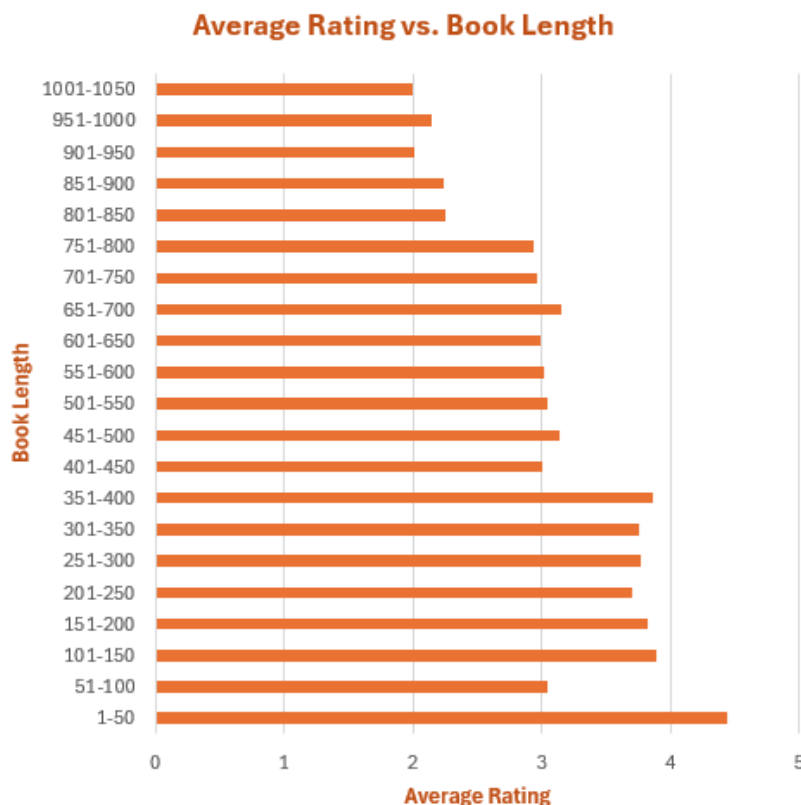
Data was extracted from the database with a specific query for each hypothesis. For the first hypothesis, the amount of pages from each book was gathered and grouped into page ranges in groups

of 50 pages. From there, the book ratings were gathered for each of these groups and the average rating was computed. This query used the book table and the user\_book\_rating table.

For the second hypothesis, the number of authors was collected from each book and grouped by that count. From there, the average rating was determined for each count of authors and modified to only look at books from June, July, and August. This query used the book table, the book\_author table, user\_book\_rating table, and the user\_book\_session table, resulting in a complex query.

### 4.3 Data Analytics & Visualization

For the first hypothesis (shorter books are rated higher), we placed our database of books into groups of 50 measured by the length of the book. We then took these groups of 50 and calculated the average of each book's rating.



However, the problem is that because the newly generated data is segmented into 50 groups, the number of books, and therefore the total number of ratings within each group, is skewed, so while the graph above seemingly proves the hypothesis, the average rating of any particular section is not necessarily comparable to the average ratings of the other sections.

For our second hypothesis (Books with multiple authors will have a higher average rating in summer months), we generated a query of all books and grouped them by the number of authors they have. We then filtered the data even more by adding the condition that the ratings collected from those books must have been rated in the summer months (June, July, August). Once we gathered all of the ratings given by those books, we simply took their averages across each author quantity. If the average rating generated increases with respect to the number of authors, the hypothesis will prove correct.

## NUMBER OF AUTHORS VS. AVG. RATING



### 4.4 Conclusions

As mentioned in the previous section, the first hypothesis is neither proven nor disproven by the graph, as the number of ratings within each group of 50 varies.

Because the graph for the second hypotheses shows no increase in average rating between author groups, the second hypothesis is not true.

Use this section to explain the conclusions drawn from your data analysis.

## 5 Lessons Learned

The biggest thing we struggled with was generating our data. We used Mockaroo for the entity tables (Book, Users, Editor, Author, Publisher, and Genre), which took some trial and error to make sure all the data was the correct type, timestamp vs date. This resulted in us generating multiple incorrect tables until we figured out the formatting correctly. However, generating these tables became much easier once we figured out the correct data types. When we needed to generate the tables that relied on other tables' data, i.e., user\_book\_session or book\_publisher, we generated the table data in Mockaroo and then used update SQL statements to ensure the data was correct and followed the proper checks. At first, we initially attempted to use Mockaroo and did not update the data afterward; however, this resulted in a large majority of our data not following the proper checks we implemented. Once we wrote and ran the update SQL statements, the data was cleaned up and followed our checks. We did attempt to use ChatGPT to help us in data generation however we ran into issues so, we stuck with Mockaroo and update scripts.

Another part of this project that we ran into trouble with was writing the complex queries needed for the application requirements. Breaking down the steps of what the queries needed to do helped us understand the scope of the queries we wrote much more easily and what should go into each script.

## 6 Tools

The tools we used in our project include Mockaroo for generating data, DataGrip for our SQL database, Java to display the data to a user through a console application, and Excel to visualize the data into graphs.